Directed Graphs; Interval Scheduling

1. Directed Graphs
   - representing directed graphs
   - search algorithms and strong connectivity
2. Directed Acyclic Graphs
   - topological ordering
   - every DAG has a topological ordering
3. Greedy Algorithms
   - interval scheduling
   - a greedy algorithm
   - the interval partitioning problem

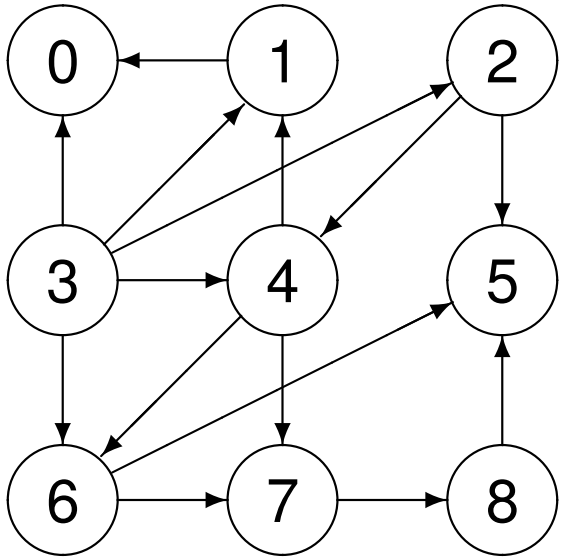
CS 401/MCS 401 Lecture 5
Computer Algorithms I
Jan Verschelde, 26 June 2026

Directed Graphs; Interval Scheduling

1. Directed Graphs
   - representing directed graphs
   - search algorithms and strong connectivity
2. Directed Acyclic Graphs
   - topological ordering
   - every DAG has a topological ordering
3. Greedy Algorithms
   - interval scheduling
   - a greedy algorithm
   - the interval partitioning problem

directed graphs

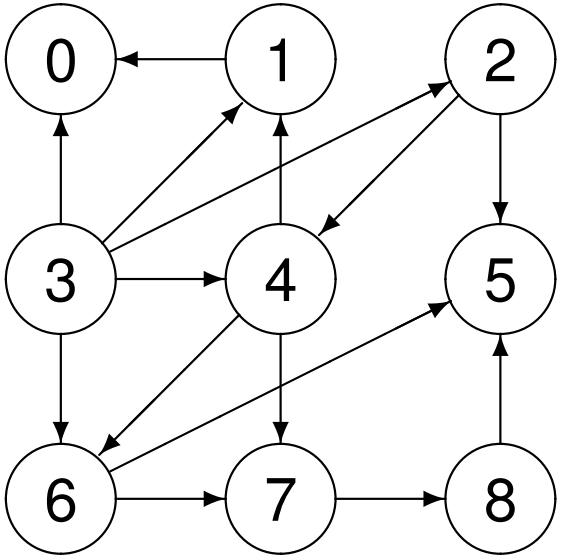
In a directed graph, every edge has an orientation.

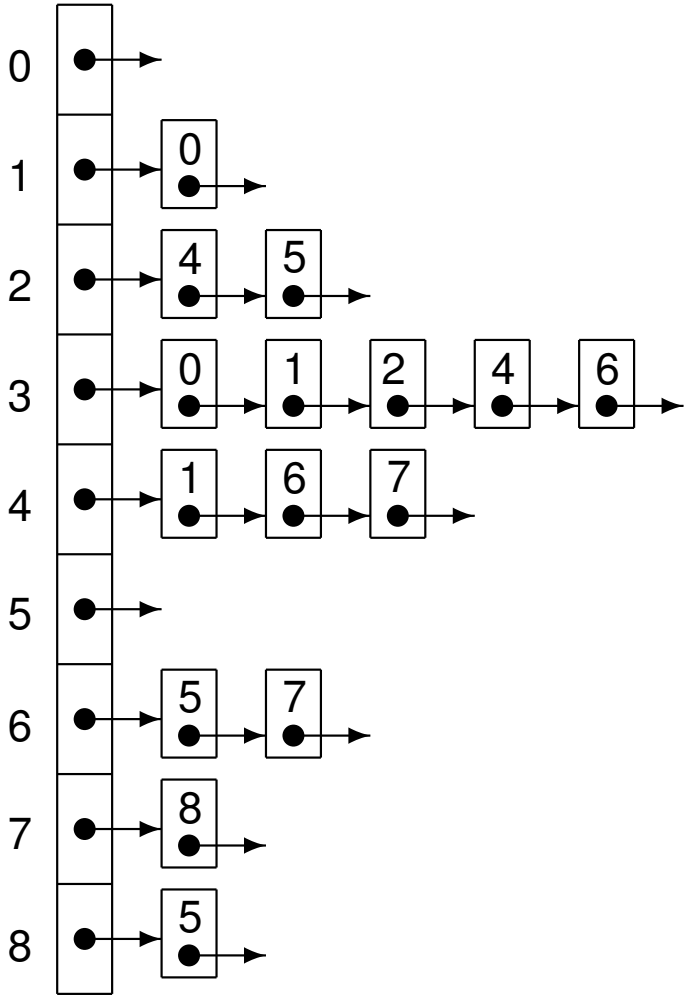


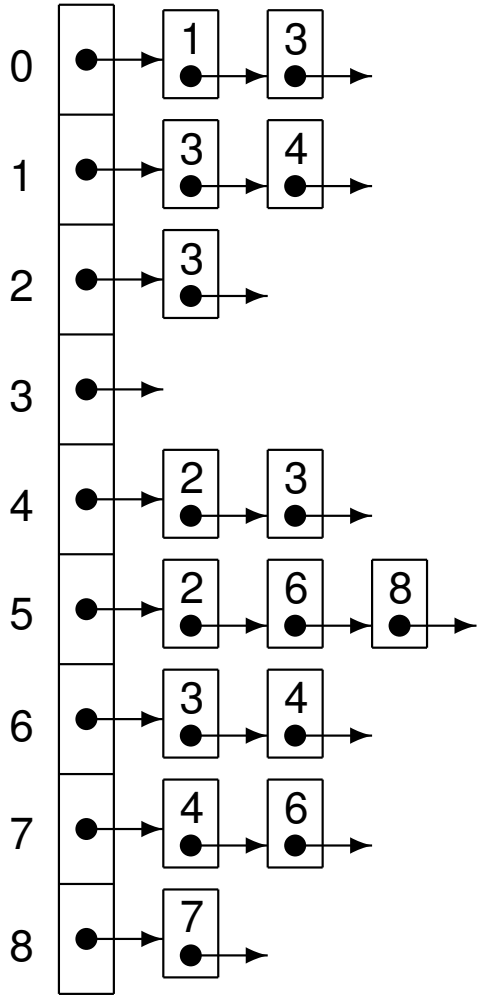
$$V = \{0, 1, 2, 3, 4, 5, 6, 7, 8\}$$

$$\begin{aligned} E \quad = \quad & \{(1,0), (3,0), (3,1), (4,1), (3,2), (2,4), (3,4), (2,5), \\ & (6,5), (8,5), (3,6), (4,6), (4,7), (6,7), (7,8)\} \end{aligned}$$

adjacency list representation of a directed graph







For every vertex, we have two lists:

(1) vertices *to which* it has edges

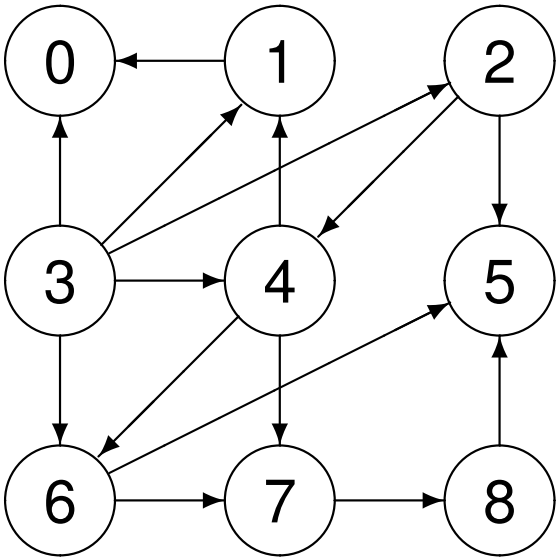
(2) vertices *from which* it has edges

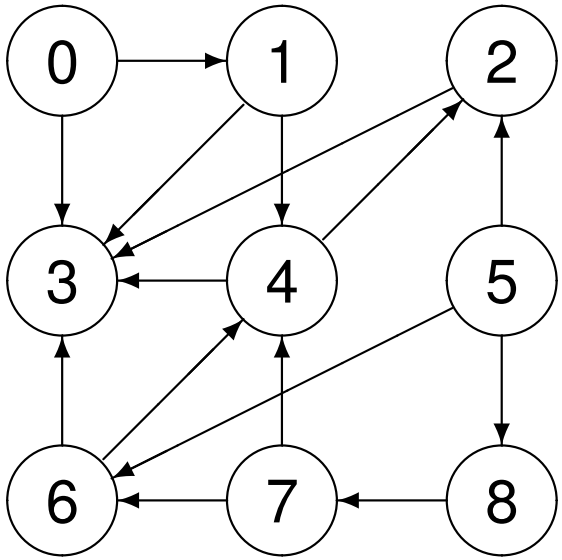
reverting the orientation of all edges

For every vertex, we have two lists:

1. the vertices *to which* it has edges; and
2. the vertices *from which* it has edges.

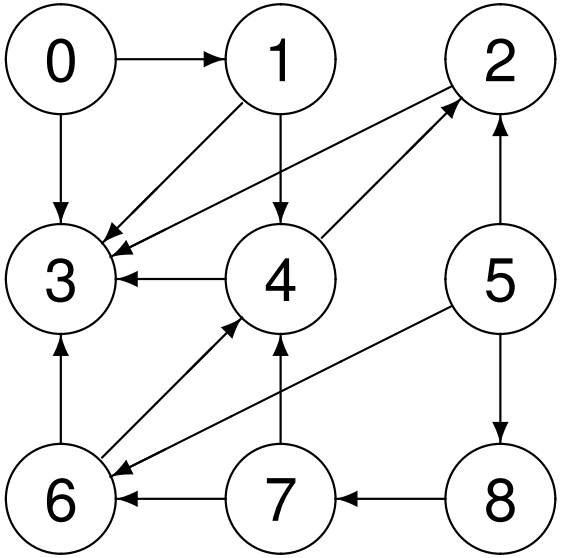
Consider reverting the orientation of all edges in a graph:

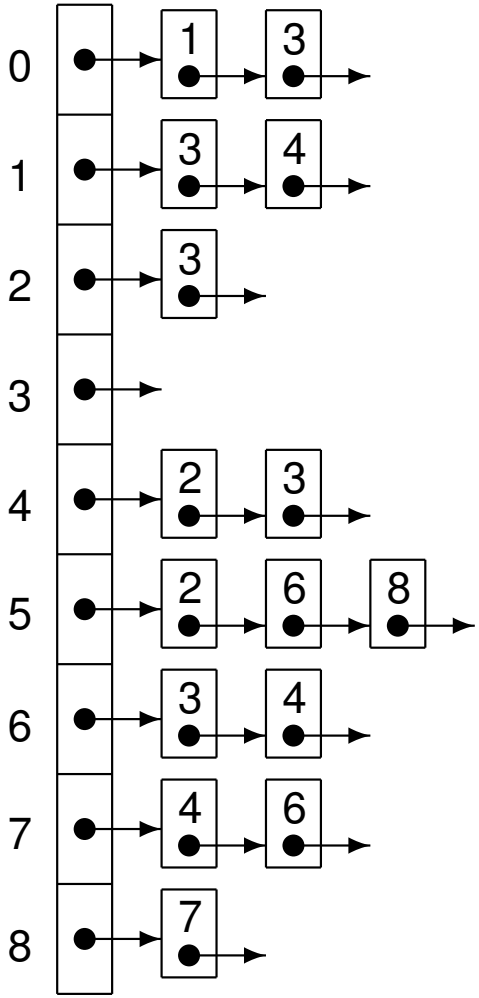


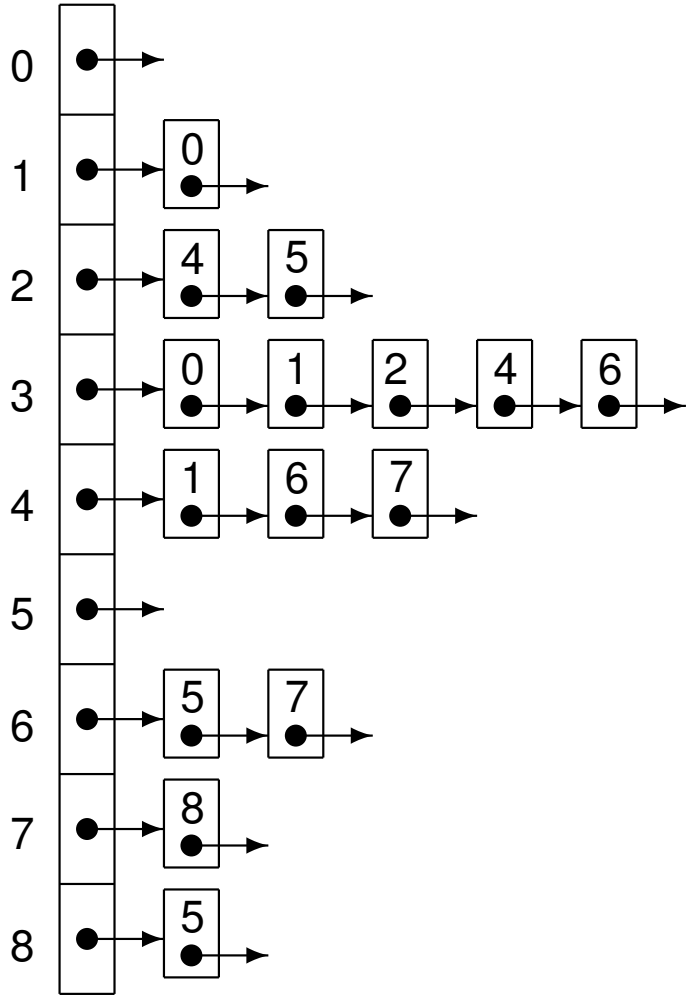


Changing the representation for the reverted graph is a simple swap.

adjacency list representation of the reverted graph







For every vertex,
we have two lists:

(1) vertices *to which* it has edges

(2) vertices *from which* it has edges

Directed Graphs; Interval Scheduling

1. Directed Graphs
   - representing directed graphs
   - search algorithms and strong connectivity
2. Directed Acyclic Graphs
   - topological ordering
   - every DAG has a topological ordering
3. Greedy Algorithms
   - interval scheduling
   - a greedy algorithm
   - the interval partitioning problem

search algorithms

Both breadth-first search and depth-first search algorithms work for directed graphs, but not every vertex which has a path from the root node has a path back to the root node.

Definition (1)

Two vertices u and v in a directed graph are *mutually reachable* if there is a path from u to v and a path from v to u.

Proposition (2)

If u and v are mutually reachable and v and w are mutually reachable, then u and w are mutually reachable.

strong connectivity and strong components

Definition (3)

A directed graph is *strongly connected* if every pair of vertices is mutually reachable.

Definition (4)

For a vertex v in a directed graph G, *the strong component containing v* is the set of all the vertices w of G such that v and w are mutually reachable.

Theorem (5)

For any two vertices v and w in a directed graph, their strong components are either identical or disjoint.

detecting strong connectivity

Exercise 1:

Consider a directed graph G with n vertices and m edges.

Is there an algorithm to detect if G is strongly connected which runs in $O(n+m)$ time?

- If not, give the argument why detecting strong connectivity must have a higher cost than graph traversal.
- If yes, prove that graph traversal algorithms can detect strong connectivity in $O(n+m)$ time.

Directed Graphs; Interval Scheduling

1. Directed Graphs
   - representing directed graphs
   - search algorithms and strong connectivity
2. Directed Acyclic Graphs
   - topological ordering
   - every DAG has a topological ordering
3. Greedy Algorithms
   - interval scheduling
   - a greedy algorithm
   - the interval partitioning problem

directed acyclic graphs and topological orderings

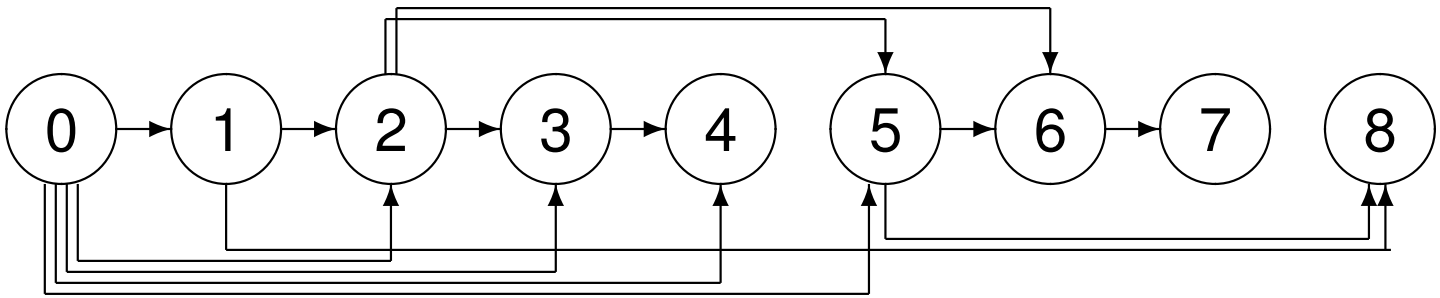
Definition (6)

A *Directed Acyclic Graph (DAG)* is a directed graph without cycles.

Definition (7)

A *topological ordering* for a directed graph is a labeling of the vertices so that for every edge (v_i, v_j) we have $i < j$.

Example:



topological ordering implies DAG

Theorem (8)

If a graph G has a topological ordering, then G is a DAG.

Proof. We prove by contradiction. Assume G has a topological ordering and G not a DAG: G has a cycle C.

Let $v_i \in C$ be the vertex with the lowest index in C.

Because C is a cycle, there a $v_j \in C$, such that

- (v_j, v_i) is an edge; and
- $j > i$, because v_i is the vertex with the lowest index in C.

In a topological ordering, if (v_j, v_i) is an edge,
then we must have that $j < i$, which leads to a contradiction.

So our assumption is false,
a topological ordering on G implies G is a DAG. Q.E.D.

a source in a DAG

Theorem (9)

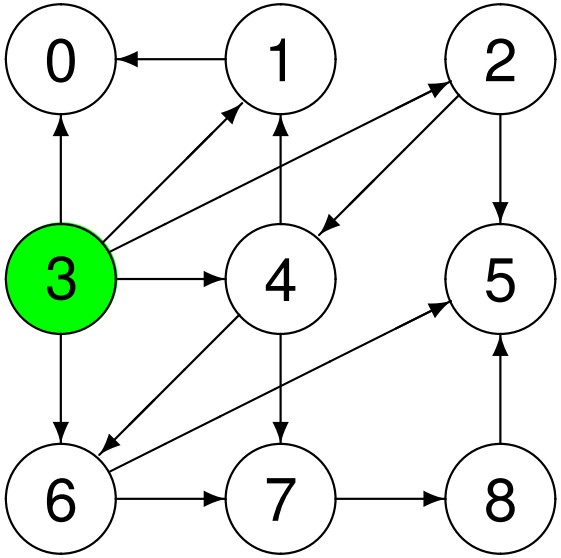
In every DAG G, there is vertex with no incoming edges.

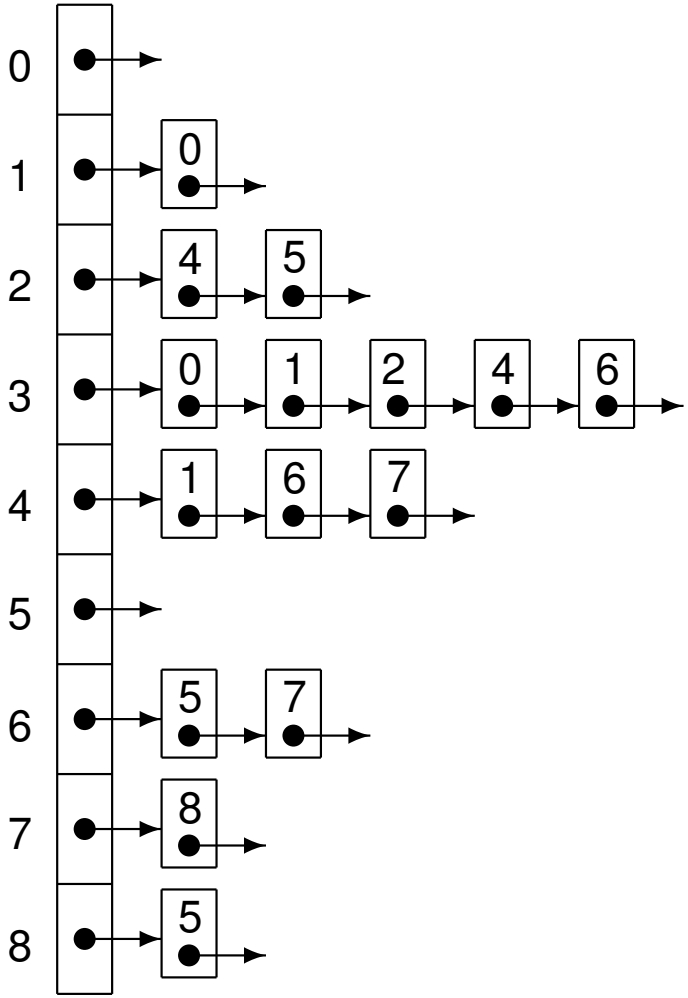
Proof. We prove by contradiction.
Assume every vertex has at least one incoming edge.
The contradiction then follows from the existence of a cycle.

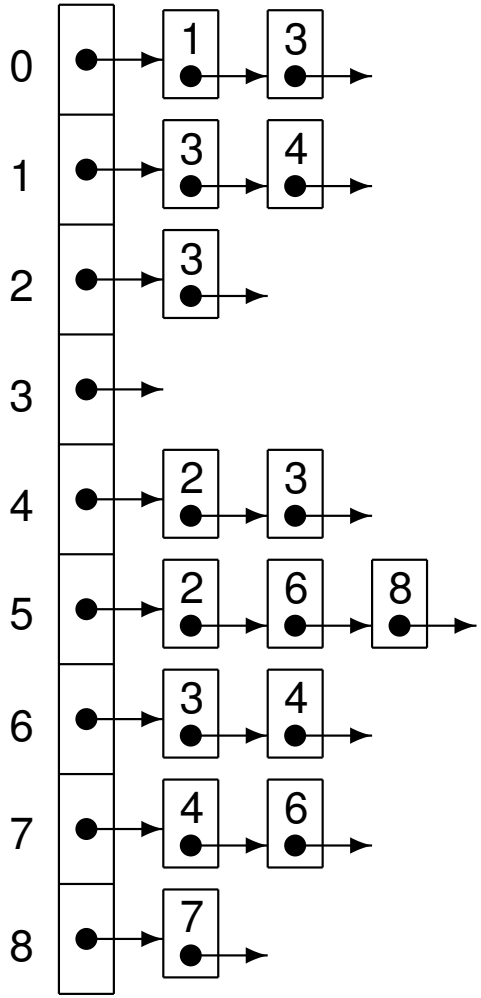
Let v be any vertex in G.
By the assumption v has at least one incoming edge (u, v).
Also u has at least one incoming edge: (t, u), and so does t, etc...
We can continue the walk indefinitely, but we will have visited some vertex w twice because G has only finitely many vertices.
The walk which contains w twice contains a cycle, which means that G is not a DAG, which contradicts that the given G is a DAG.

So our assumption that every vertex has at least one incoming edge is false and there is a vertex with no incoming edges. Q.E.D.

finding a source in a DAG







For every vertex, we have two lists:

(1) vertices *to which* it has edges

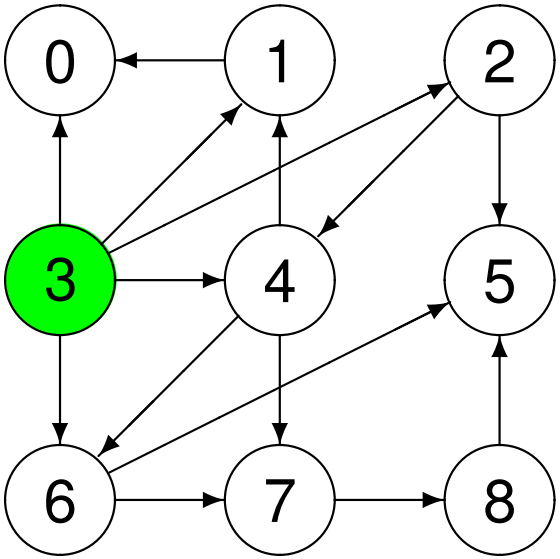
(2) vertices *from which* it has edges

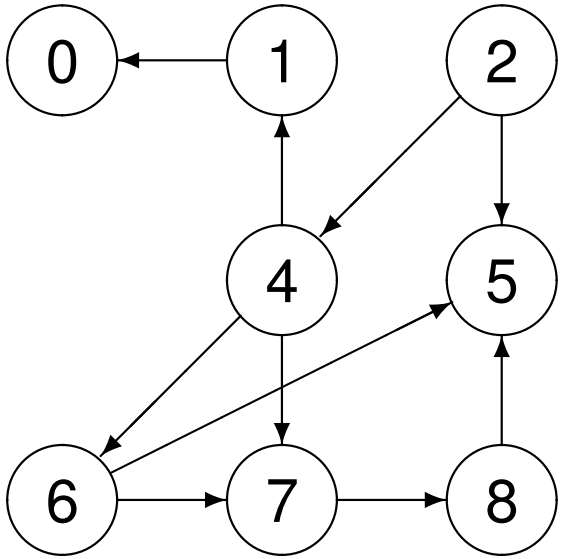
a recursive algorithm

To compute a topological ordering of G

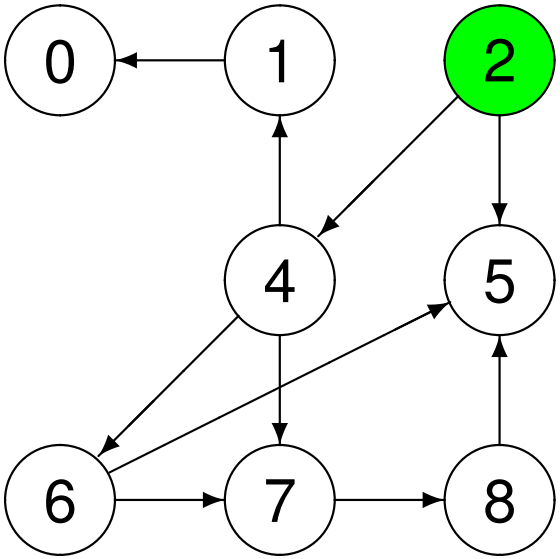
find a vertex v in G with no incoming edges
place v first
delete v from G
recursively compute a topological ordering of $G \setminus \{v\}$
and append this order after v

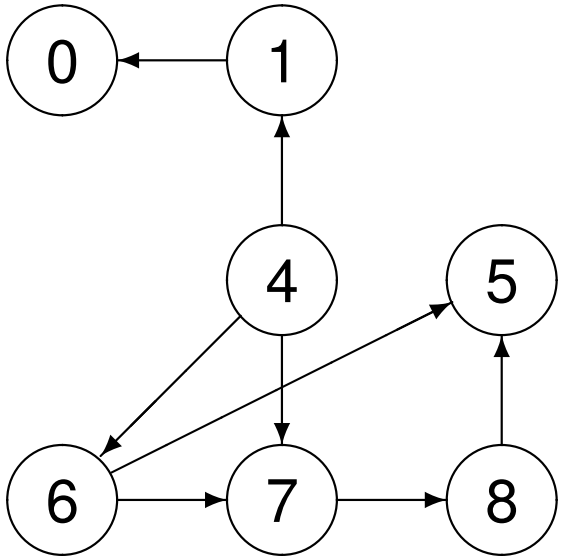
deleting the source 3



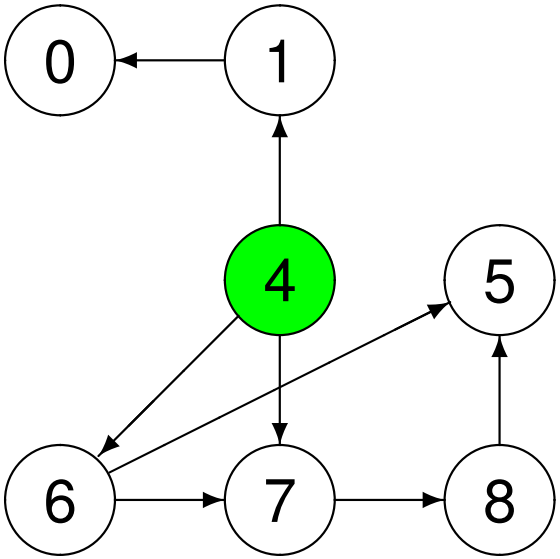


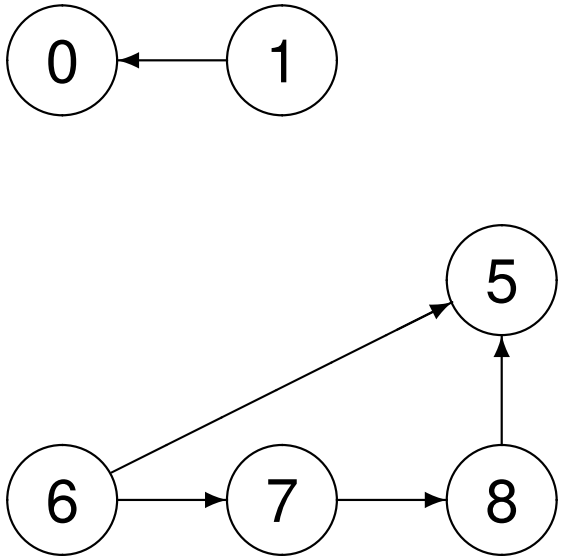
finding and deleting the source 2



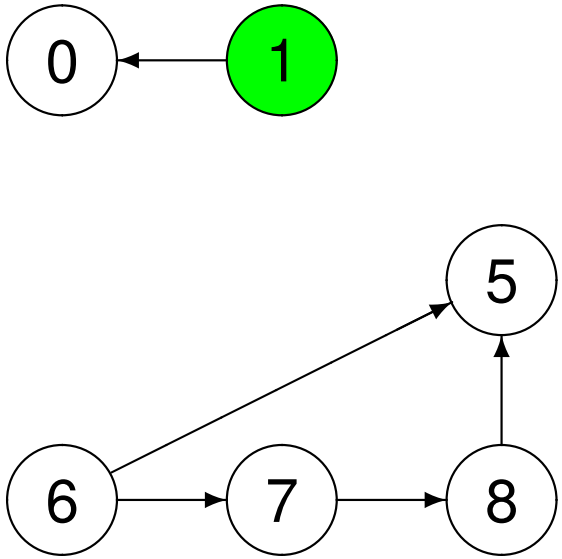


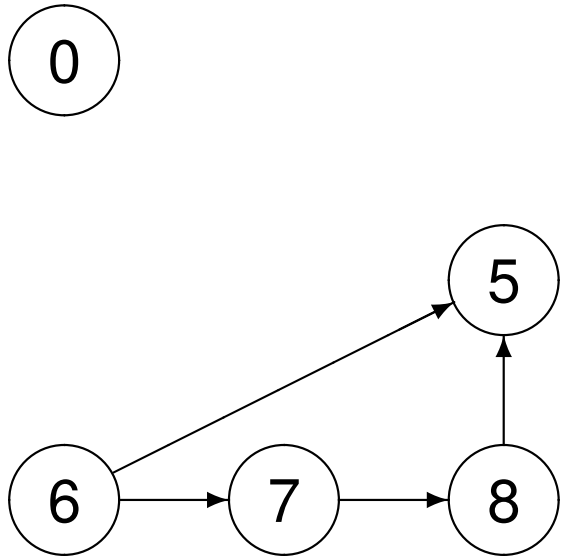
finding and deleting the source 4



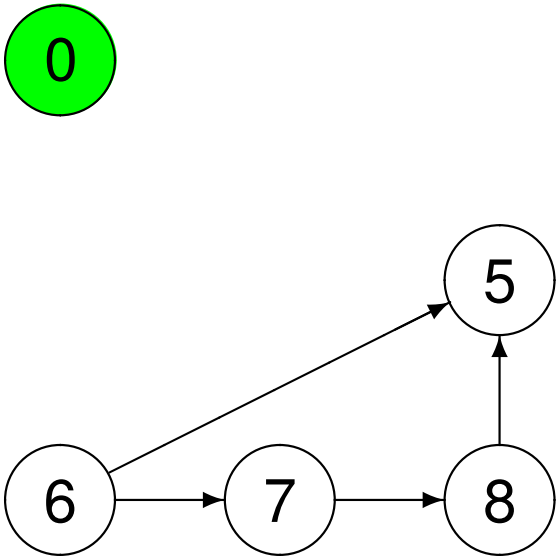


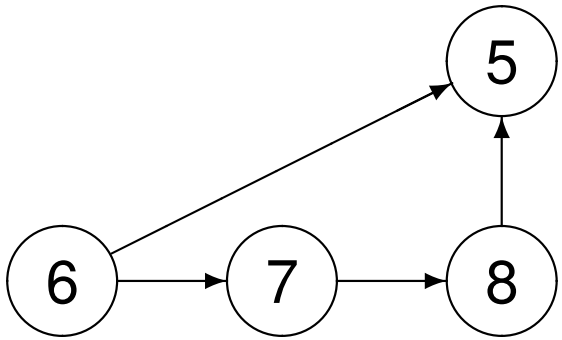
finding and deleting the source 1



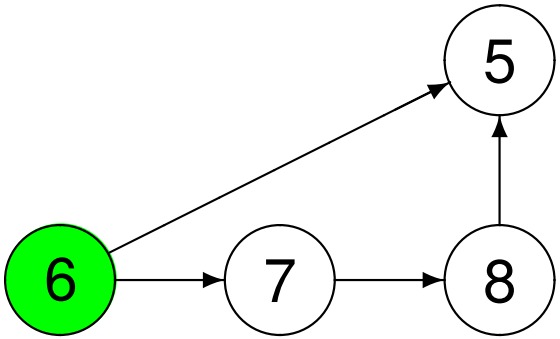


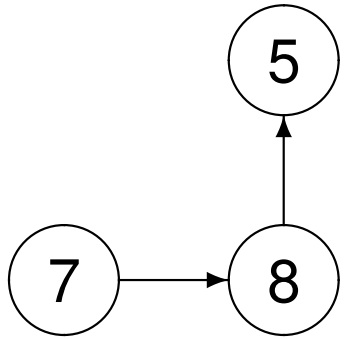
finding and deleting the source 0



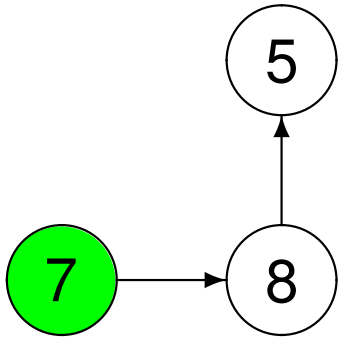


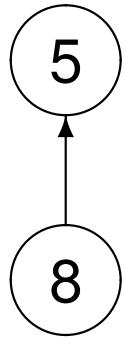
finding and deleting the source 6



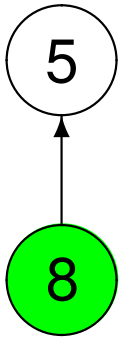


finding and deleting the source 7

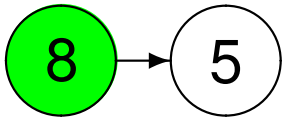




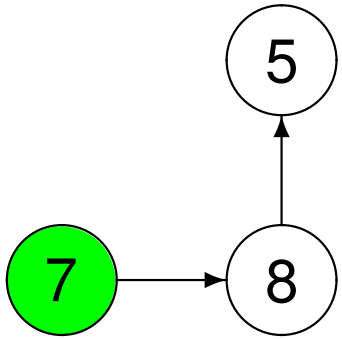
appending the topological order after 5

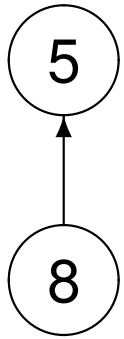


append 5 after 8:

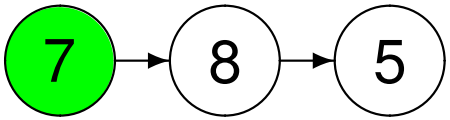


appending the topological order after 7

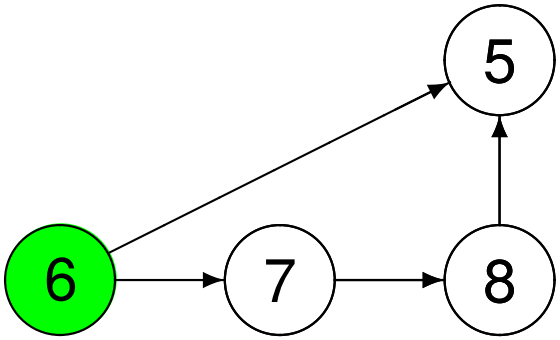


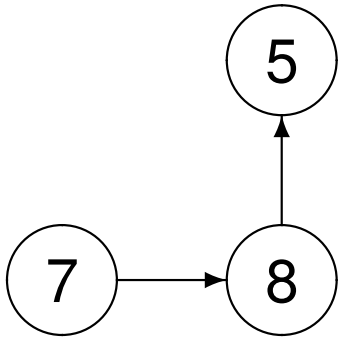


append the topological order after 7:

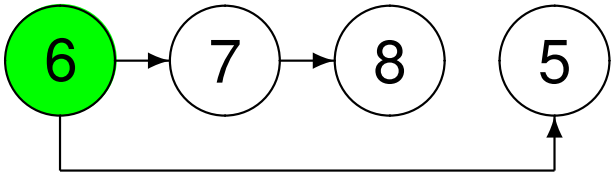


appending the topological order after 6

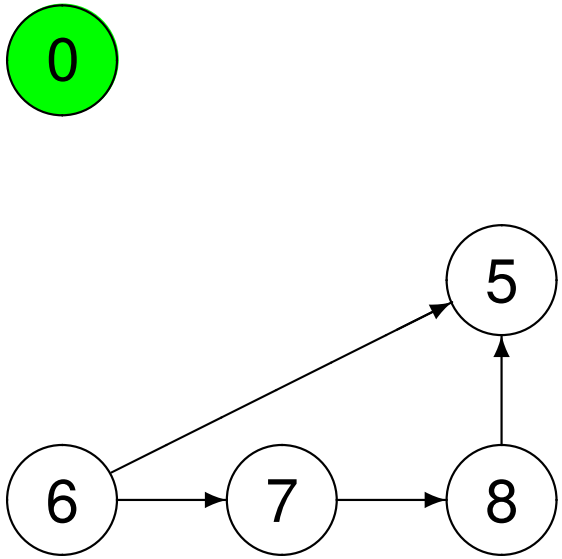


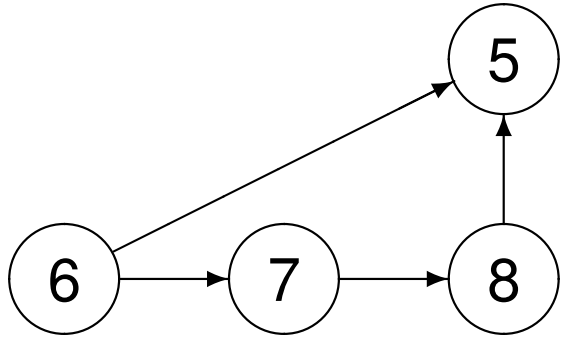


append the topological order after 6:

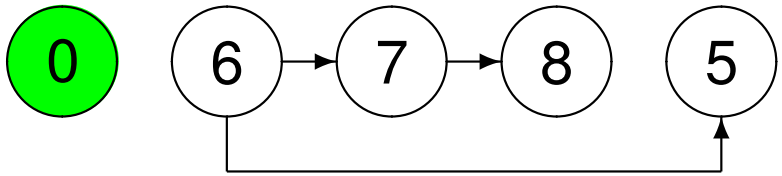


appending the topological order after 0

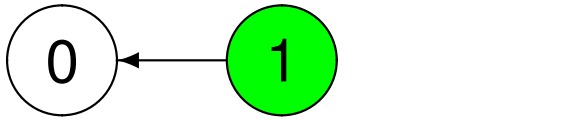


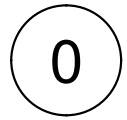


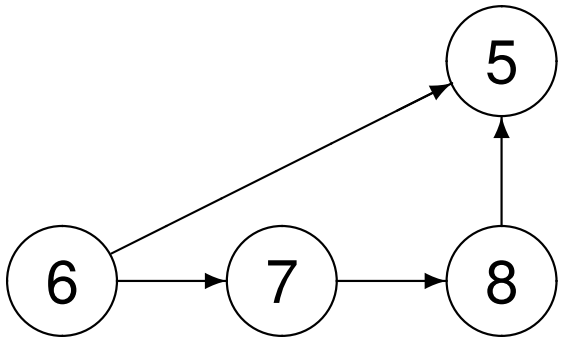
append the topological order after 0:

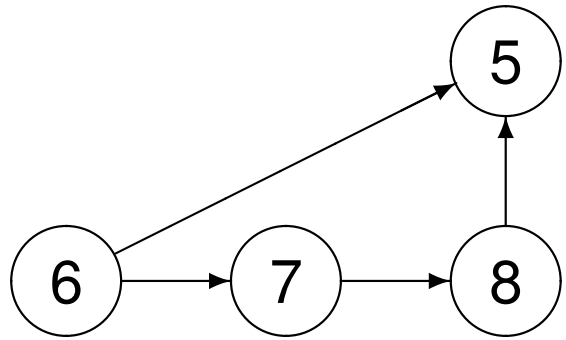


appending the topological order after 1

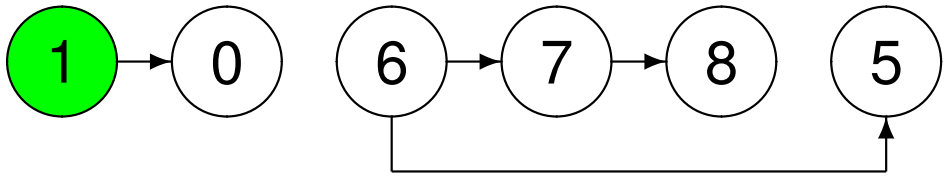




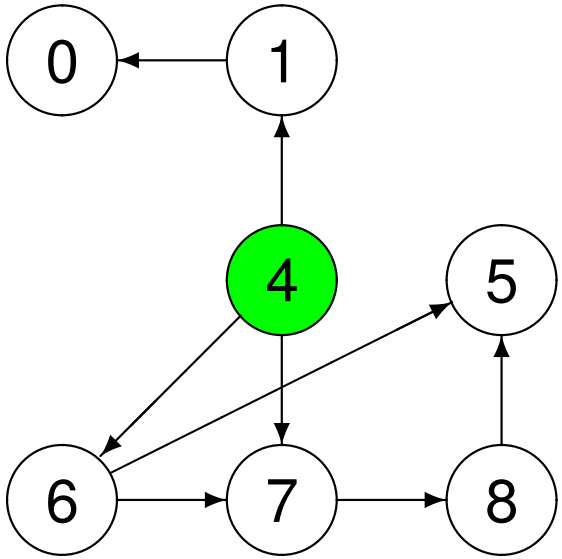


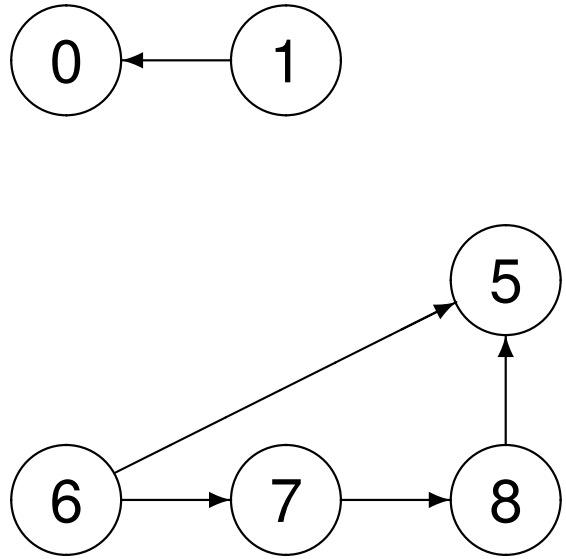


append the topological order after 1:

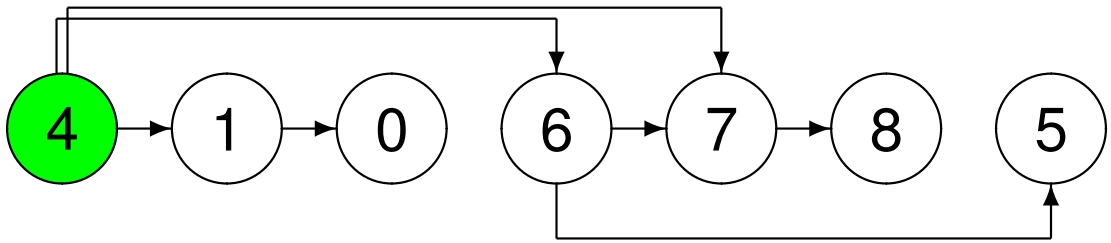


appending the topological order after 4

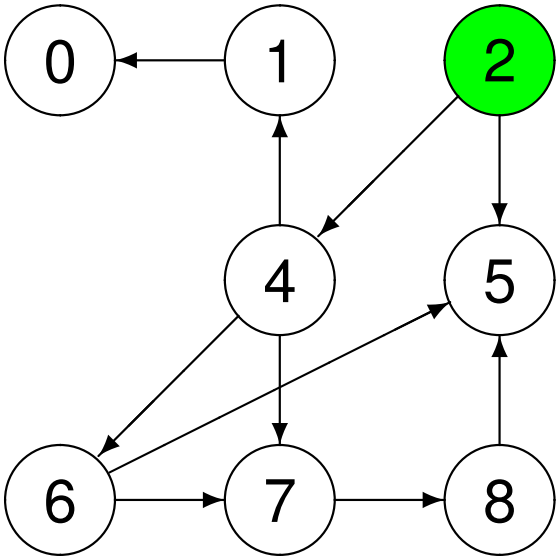


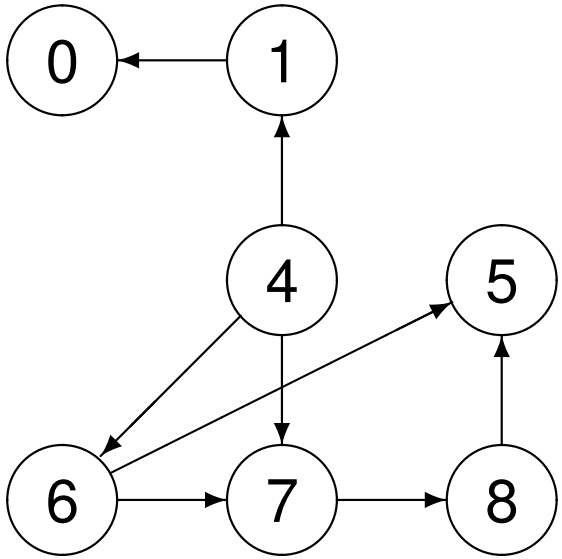


append the topological order after 4:

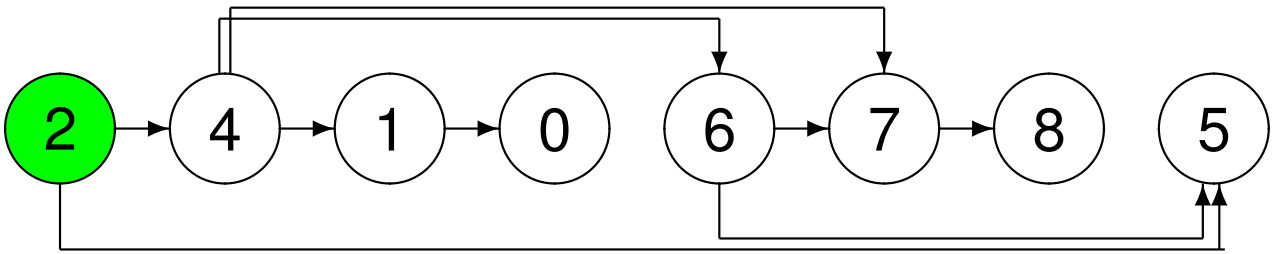


appending the topological order after 2

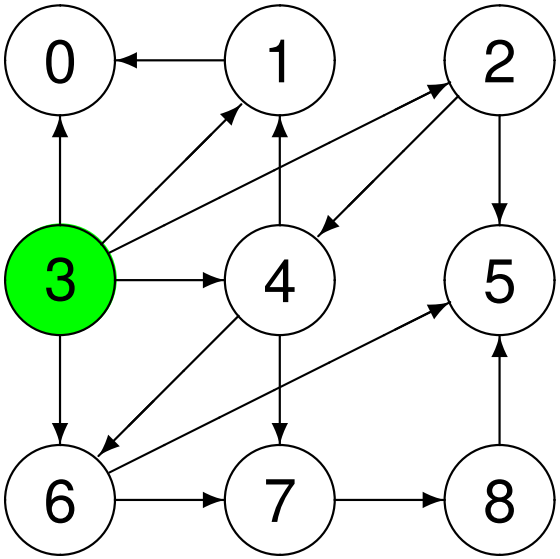


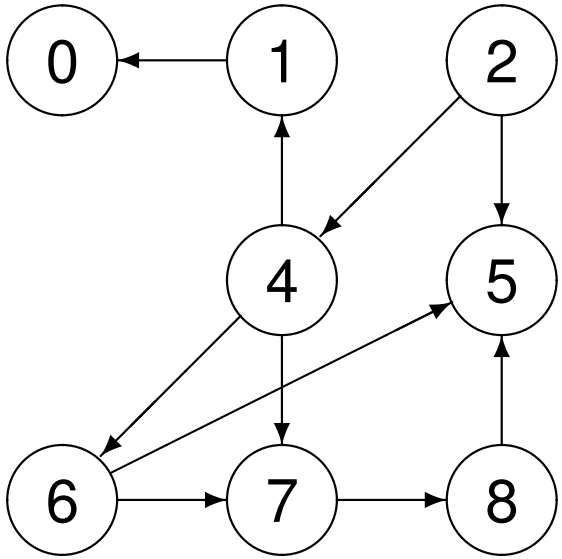


append the topological order after 2:

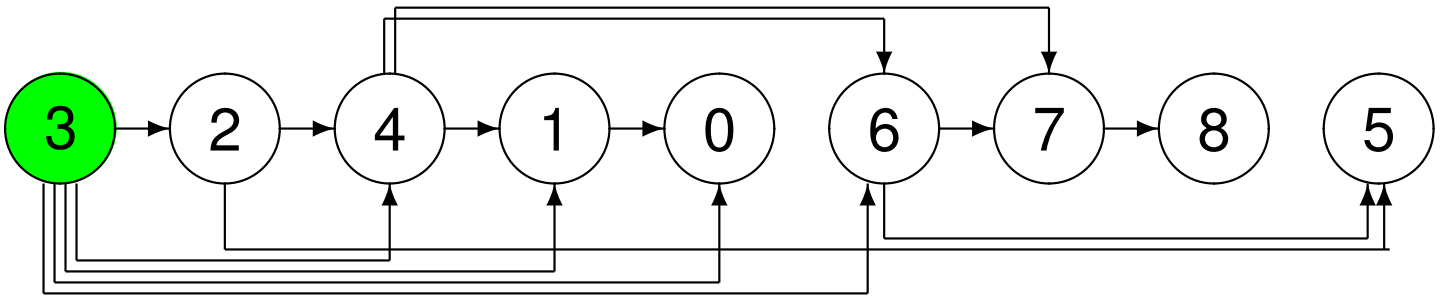


appending the topological order after 3

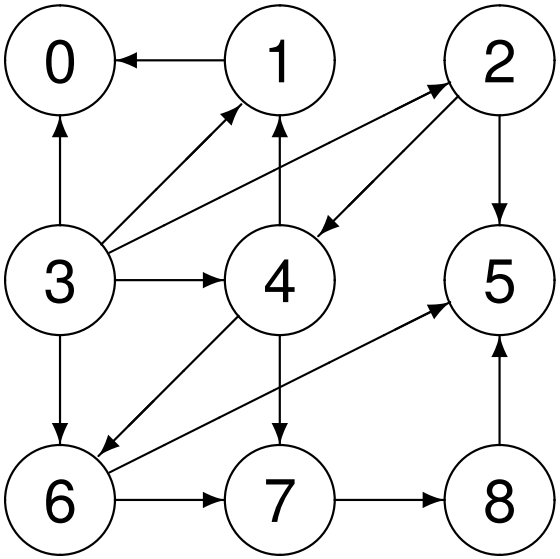


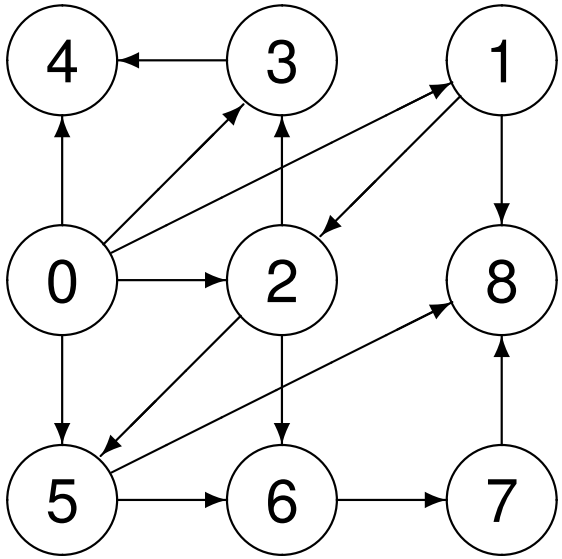


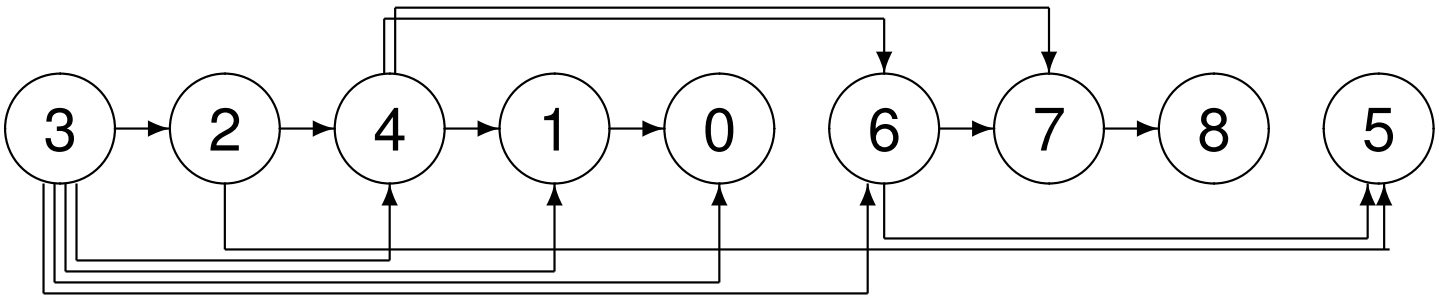
append the topological order after 3:



relabeling the vertices





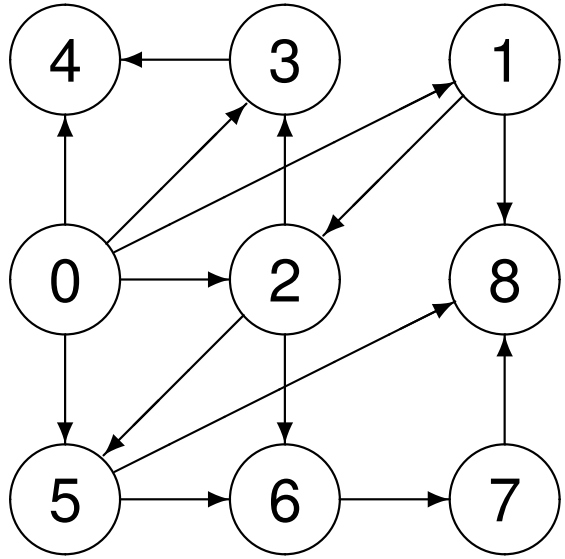


two exercises

Exercise 2:

Consider the graph G:

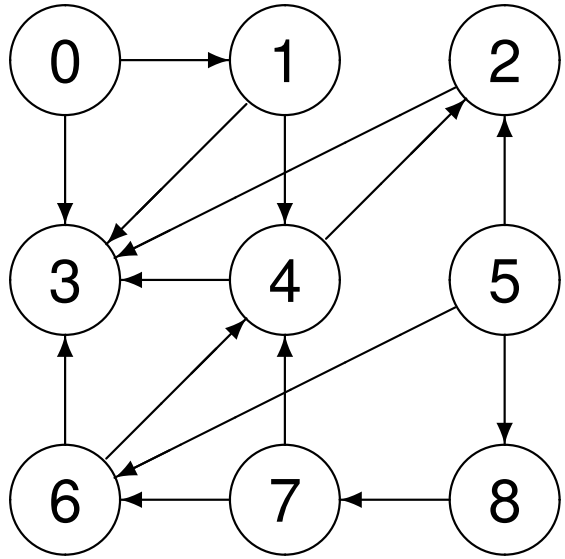
Write the adjacency list representation for G.



Exercise 3:

Consider the graph G:

Run the topological ordering algorithm on G. Show all steps in the algorithm.



Directed Graphs; Interval Scheduling

1. Directed Graphs
   - representing directed graphs
   - search algorithms and strong connectivity
2. Directed Acyclic Graphs
   - topological ordering
   - every DAG has a topological ordering
3. Greedy Algorithms
   - interval scheduling
   - a greedy algorithm
   - the interval partitioning problem

every DAG has a topological ordering

Theorem (10)

If G is a DAG, then G has a topological ordering.

Proof. We prove by induction on the number of vertices.
Base case: the statement is true for a graph with one vertex.
Induction hypothesis: it is true for all graphs with n vertices.
Assuming this hypothesis, we have to show it is true for $n + 1$ vertices.

Consider G a DAG with $n + 1$ vertices.
Because G is a DAG, there is a vertex v with no incoming edges.
Consider $G \setminus \{v\}$:

1. $G \setminus \{v\}$ has no cycles because G is a DAG and has no cycles and the removal of v (and its incoming edges) did not introduce any new edges, and thus no cycles.
   Therefore, $G \setminus \{v\}$ is a DAG.

proof continued

2. $G \setminus \{v\}$ has n vertices and the induction hypothesis applies.

   We apply the induction hypothesis to $G \setminus \{v\}$:
   $G \setminus \{v\}$ has a topological ordering.

   All vertices (and their edges) in the topological ordering of $G \setminus \{v\}$ are appended to v (and its edges),
   so we end up with a topological ordering for G.

   G has $n + 1$ vertices and a topological ordering,
   which proves the induction hypothesis. Q.E.D.

cost of the topological order algorithm

Identifying a vertex with no incoming edges with a graph with n vertices can be done in $O(n)$ time.

The algorithm runs for n iterations,
each step requires the finding of a source vertex, which takes $O(n)$,
so the total running time is $O(n^2)$, which is okay for a dense graph.

The cost can be reduced to $O(n+m)$, where m is the number of edges of the graph, keeping track of active vertices.

A vertex is *active* if it has not yet been deleted.

- Store for each vertex w, the number of incoming edges from active vertices; and
- store the set S of all active nodes that have no incoming edges from other active vertices.

The amount of work per edge is constant during the algorithm.

overview of algorithms on graphs we covered

We close chapter 3 on Graphs.

1. Breadth-first search.
2. Depth-first search.
3. Computing the set of connected components.
4. Testing if a graph bipartite.
5. Topological ordering.

Directed Graphs; Interval Scheduling

1. Directed Graphs
   - representing directed graphs
   - search algorithms and strong connectivity
2. Directed Acyclic Graphs
   - topological ordering
   - every DAG has a topological ordering
3. Greedy Algorithms
   - interval scheduling
   - a greedy algorithm
   - the interval partitioning problem

CS 401/MCS 401 Lecture 5
Computer Algorithms I
Jan Verschelde, 26 June 2026

Directed Graphs; Interval Scheduling

1. Directed Graphs
   - representing directed graphs
   - search algorithms and strong connectivity
2. Directed Acyclic Graphs
   - topological ordering
   - every DAG has a topological ordering
3. Greedy Algorithms
   - interval scheduling
   - a greedy algorithm
   - the interval partitioning problem

interval scheduling

Our problem is to schedule requests for a resource.

Input: n requests $\{ 1, 2, \ldots, n \}$,
each request is a time interval $[s(i), f(i)]$, where
$s(i)$ is the starting time of the i-th request; and
$f(i)$ is the finishing time of the i-th request, $f(i) > s(i)$.

Output: a subset S of $\{ 1, 2, \ldots, n \}$ so that
no two intervals in S overlap; and
$\#S$ is as large as possible.

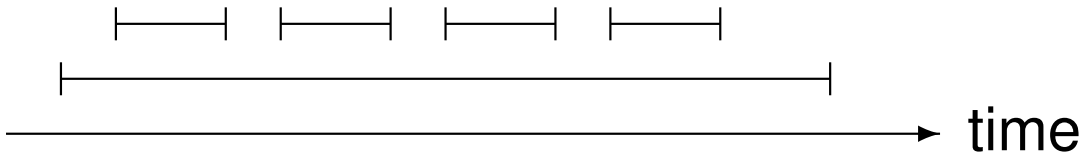
The goal is to *schedule as many requests as possible.*

obvious rule: select request that starts earliest

We want the resource being used as soon as possible.

We select the request i with the smallest starting time $s(i)$.

Consider the following input:



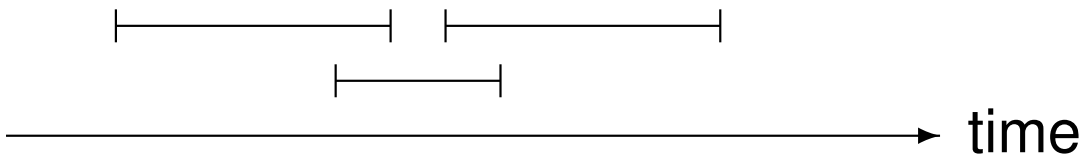
In the above input, we select only one request, not optimal:
we should have selected the four other requests.

second rule: select request with shortest interval

The previous example suggests we better select the smallest interval.

We select the request i for which $|f(i) - s(i)|$ is minimal.

Consider the following input:



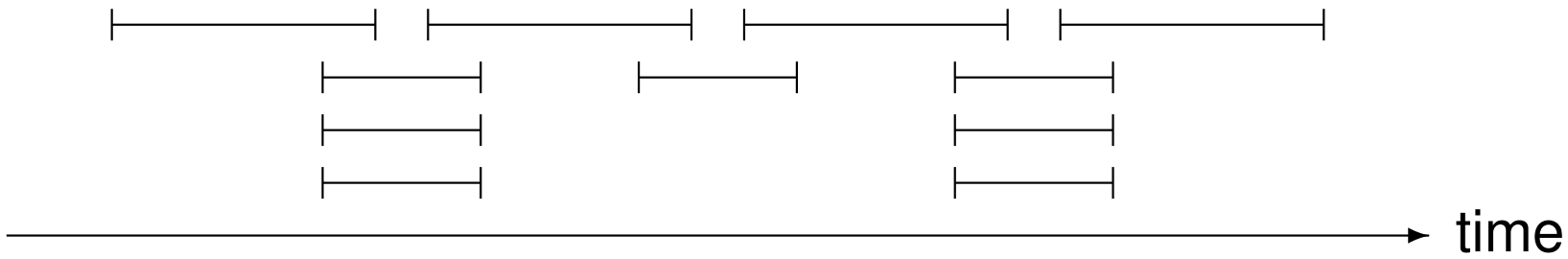
In the above input, we select only one request, not optimal:
we should have selected the two other requests.

third rule: select request with fewest conflicts

The previous example suggests we better minimize conflicts.

We select the request for which its time interval overlaps with the fewest number of intervals.

Consider the following input:



In the above input, we select the middle interval which prevents two other intervals from being scheduled.

With this rule, we can schedule only three requests, while we could schedule four requests.

Directed Graphs; Interval Scheduling

1. Directed Graphs
   - representing directed graphs
   - search algorithms and strong connectivity
2. Directed Acyclic Graphs
   - topological ordering
   - every DAG has a topological ordering
3. Greedy Algorithms
   - interval scheduling
   - a greedy algorithm
   - the interval partitioning problem

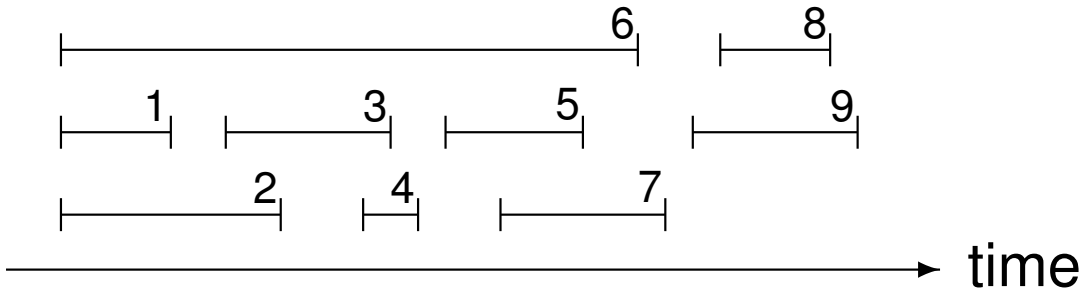
a greedy algorithm

Another (fourth) idea: *select the request that finishes first*,
as we want our resource to become free as soon as possible.

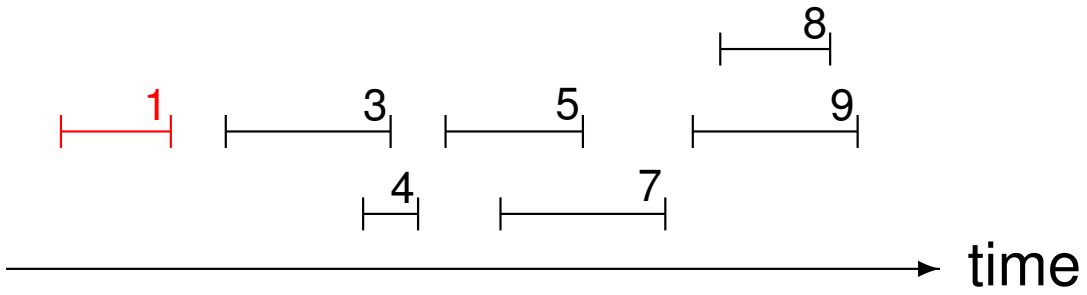
A more formal description of the algorithm is below:

let R be the set of all requests
let $\mathcal{A}$ be the set of accepted requests
while $R \neq \emptyset$ do
    let $i \in R$ that has the smallest $f(i)$
    $\mathcal{A} := \mathcal{A} \cup \{i\}$
    remove all requests from R that overlap with i

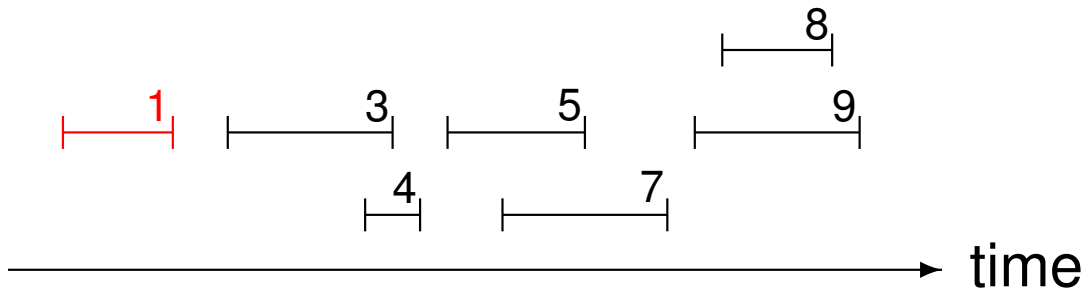
an example



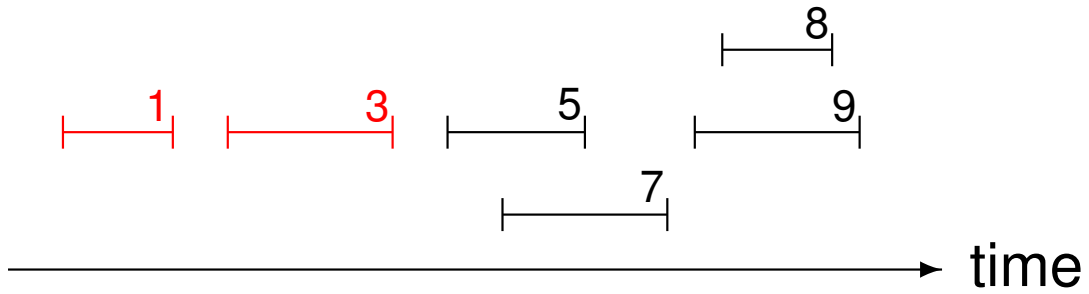
We select 1 and remove conflicting requests 2 and 6:



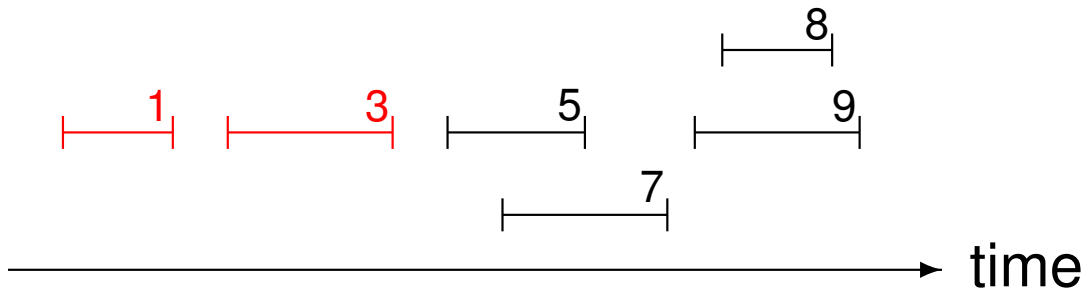
an example continued



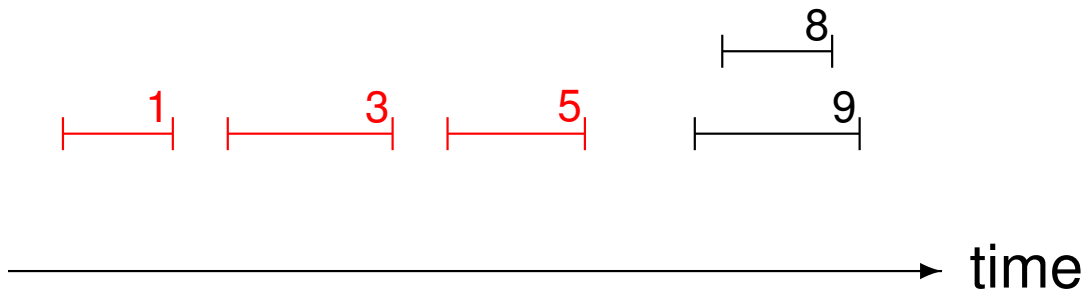
We select 3 and remove the conflicting request 4:



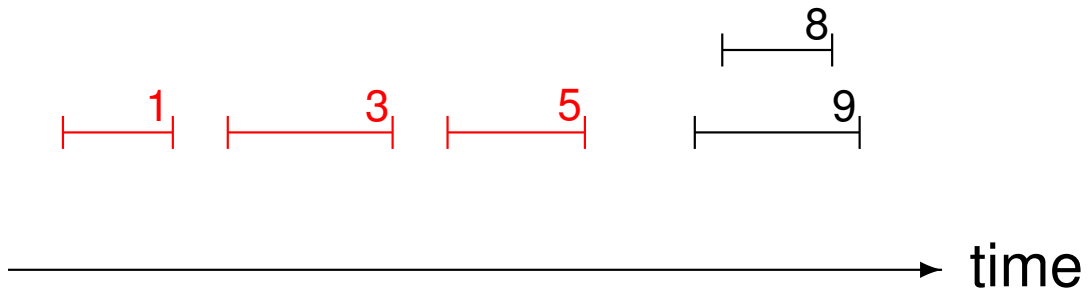
an example continued



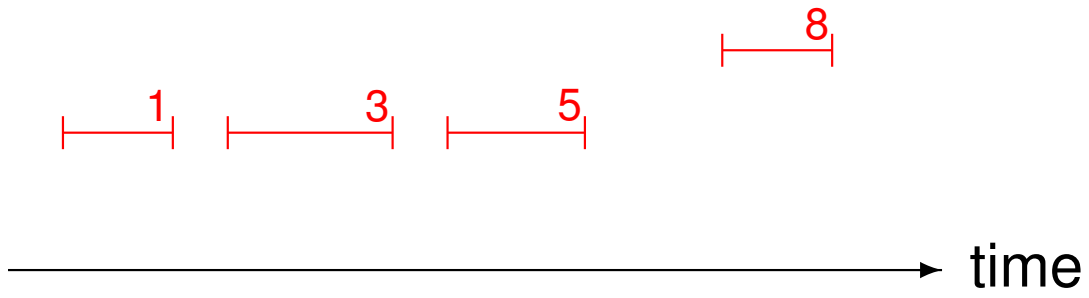
We select 5 and remove the conflicting request 7:



an example completed



We select 8 and remove the conflicting request 9:



some exercises

Exercise 4:
Run the algorithm on the input on slide #42.

Exercise 5:
Run the algorithm on the input on slide #43.

Exercise 6:
Run the algorithm on the input on slide #44.

analysis of the algorithm

$\mathcal{A}$ is the set of accepted requests, returned by the algorithm.

Denote $\mathcal{O}$ an optimal set of requests, not necessarily unique.

We want to show that $\#\mathcal{A} = \#\mathcal{O}$, the selection is optimal.

More precisely, let $\mathcal{A} = \{ i_1, i_2, \ldots, i_M \}$ and $\mathcal{O} = \{ j_1, j_2, \ldots, j_m \}$.

Our goal is to prove that $M = m$. Even as $\mathcal{O}$ is not unique, m is.

Lemma (11)

For all indices $\ell \leq k$, we have $f(i_\ell) \leq f(j_\ell)$, $i_\ell \in \mathcal{A}$, $j_\ell \in \mathcal{O}$.

The lemma states that *the greedy algorithm stays ahead*.

proof of the Lemma by induction

Lemma (11)

For all indices $\ell \leq k$, we have $f(i_\ell) \leq f(j_\ell)$, $i_\ell \in \mathcal{A}$, $j_\ell \in \mathcal{O}$.

Proof. We prove by induction on ℓ.

1. Base case: $\ell = 1$ is true because the algorithm selects the interval with the earliest finishing time.
2. Induction Hypothesis: $f(i_\ell) \leq f(j_\ell)$ for all $\ell \leq r-1$, $r > 1$.
   We need to prove for $\ell = r$.
3. The induction hypothesis implies that $f(i_{r-1}) \leq f(j_{r-1})$
   Because $\mathcal{O}$ consists of nonoverlapping intervals: $f(j_{r-1}) \leq s(j_r)$.

   Thus j_r is available when the greedy algorithm selects i_r.

   Because the greedy algorithm selects the interval with the smallest finishing time: $f(i_r) \leq f(j_r)$,
   which proves the induction hypothesis. Q.E.D.

the greedy algorithm is optimal

Theorem (12)

The greedy algorithm returns an optimal set $\mathcal{A}$.

Proof. We prove by contradiction. Assume $\mathcal{A}$ is not optimal, so $m = \#\mathcal{O} > \#\mathcal{A} = M$, for an optimal set $\mathcal{O}$.

Because $m > M$, $\mathcal{O}$ contains j_{M+1}, a nonoverlapping request with j_M.

By the Lemma (11): $f(i_M) \leq f(j_M)$, which implies that j_{M+1} did not overlap with i_M and was thus was available when the greedy algorithm stopped.

But the greedy algorithm stops only when the set of all requests $R = \emptyset$. As j_{M+1} did not overlap, we have: $j_{M+1} \in R$.

This leads to a contradiction, so our assumption is false and $\mathcal{A}$ is optimal. Q.E.D.

implementation and running time

selecting data structures

To run the algorithm on n requests in $O(n \log(n))$,
we sort the requests in order of their finishing time: $f(i) \leq f(j)$, for $i < j$.
The sorting takes $O(n \log(n))$.

We have the array $\mathtt{S}$ of size n which contains $s(i)$.
Constructing $\mathtt{S}$ takes $O(n)$.

Proceed as follows, through the ordered requests:

1. Select always the first request, because it finishes first.
2. Find the first request j for which $s(j) \geq f(1)$.
3. Select the next request after this request j,
   the request that finishes first.

One pass through the request works, with $O(1)$ for each interval.

Directed Graphs; Interval Scheduling

1. Directed Graphs
   - representing directed graphs
   - search algorithms and strong connectivity
2. Directed Acyclic Graphs
   - topological ordering
   - every DAG has a topological ordering
3. Greedy Algorithms
   - interval scheduling
   - a greedy algorithm
   - the interval partitioning problem

the interval partitioning problem

The input to the interval partitioning problem is the same as the input to the interval scheduling problem.

Instead of one single resource,
we have sufficiently many resources to satisfy all requests.

Example: schedule lectures with given start and finish times
in different classrooms.

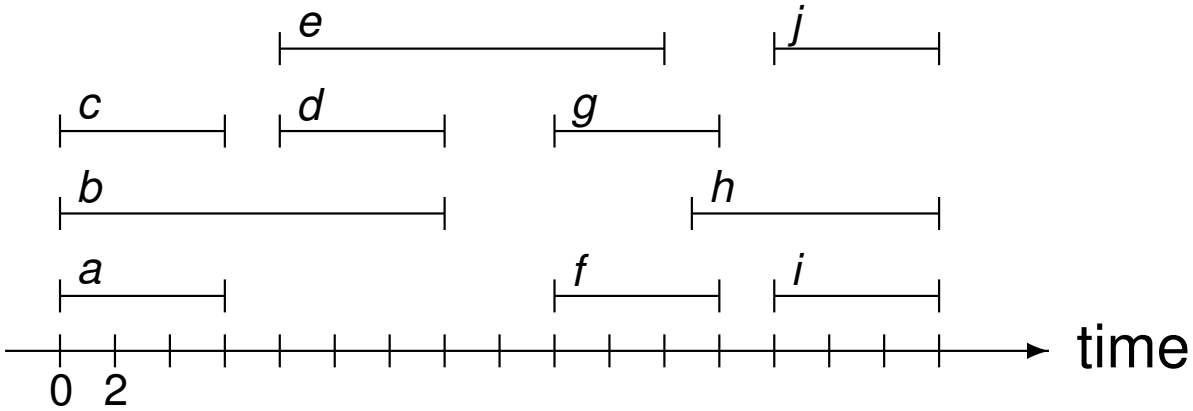
Definition (13)

The *depth* of a set of intervals is the maximum number of overlapping intervals that pass over any single point on the time line.

Proposition (14)

In any instance of the interval partitioning problem, the number of needed resources is at least the depth of the set of intervals.

an example

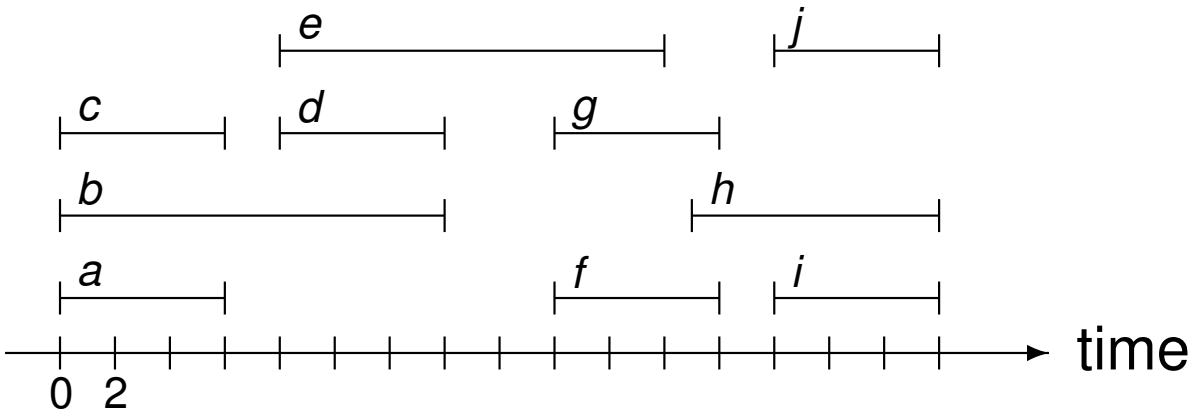


The depth of the set of intervals is also given on input.

Exercise 7:

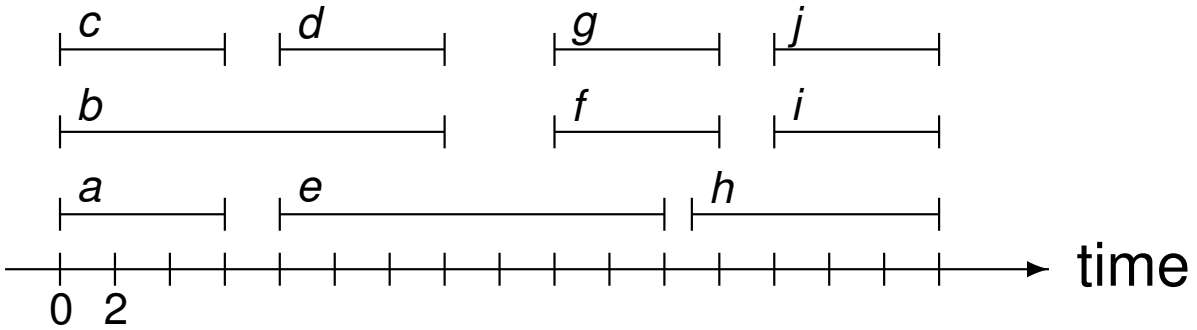
Describe an algorithm to compute the depth of a set of intervals.
Illustrate your algorithm on the example above.
What is the running time of your algorithm? Justify your answer.

the solution of the example



The depth of the set of intervals equals three.

And three resources suffice to schedule all requests:



the algorithm

1. Let $\{ \ell_1, \ell_2, \ldots, \ell_d \}$ be a set of labels,
   where d is the depth of the set of intervals.
2. The intervals are sorted by their start times.
   If two intervals start at the same time,
   then break the tie by selecting the one with the lowest index.
3. For each interval i do
   1. Reset all available labels to $L = \{ \ell_1, \ell_2, \ldots, \ell_d \}$.
   2. For each interval j that starts earlier than i
      and that overlaps with i, exclude the label given to j from L.
   3. If there is a label $\ell_k \in L$ that is not excluded,
      then label i with ℓ_k, else leave i unlabeled.

Exercise 8:
Run this algorithm on the example input of the previous slide.
Show all steps in the execution of the algorithm.

analyzing the algorithm

Lemma (15)

The greedy algorithm will label every interval
and no two overlapping intervals will receive the same label.

Proof. Consider two overlapping intervals i and j, with $s(i) \leq s(j)$, and $i < j$.

When j is considered by the algorithm, i is in the set of intervals whose labels are excluded from consideration.

Consequently, j will not be assigned to the label used for i.

Therefore, i and j are assigned to different labels. Q.E.D.

the greedy algorithm is optimal

Theorem (16)

Using as many resources as the depth of the intervals, the greedy algorithm schedules every interval.

Proof. We prove that no interval ends up unlabeled.
Then, we apply Lemma (15).

Consider the j-th interval which overlaps with t intervals.
These t intervals, jointly with j, define a set of $t + 1$ intervals that all pass over a common points on the time axis.

So, $t + 1 \leq d$ and thus $t \leq d - 1$.
Therefore, there is one label that can be assigned to j.

By Lemma (15), no two overlapping intervals receive the same label, so all intervals are scheduled.

Q.E.D.

an application

Exercise 9:

A hotel reservation manager plans the assignment of rooms to individual guests for a number of stays during the summer.

Rooms are not to be shared between different guests for any stay.

Each stay is given by a check-in day and a check-out day.

1. Describe an efficient algorithm to solve this planning problem.
2. Prove the correctness and the running time of your algorithm.